

Lecture 9

Stacks & Queues

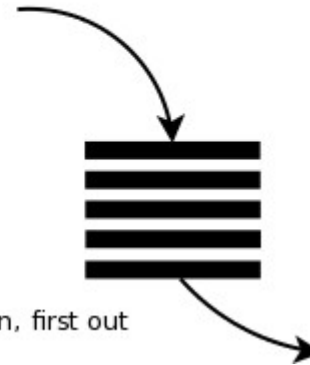
Stack:

Last in, first out



Queue:

First in, first out



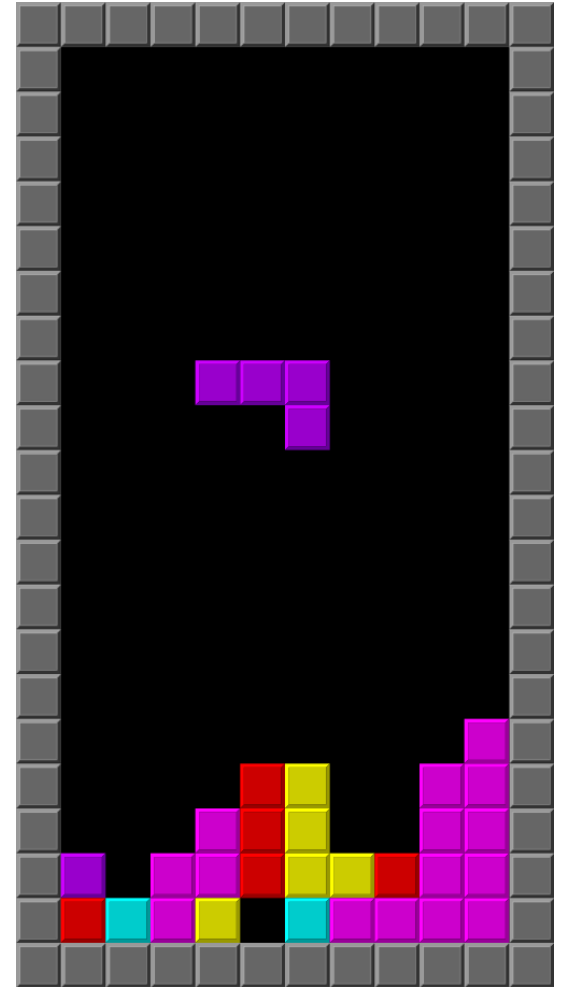
Announcements

- ❖ **Office hours** are up on course website
 - T: 12pm-4pm; F: 11am-1pm
 - Schedule an appointment individually or as a group
- ❖ **Project 2** Planning & Analysis due **Friday @ 11:59pm**
 - See [example project](#) on Github
 - Be sure to comment your Github link on the project document

Today

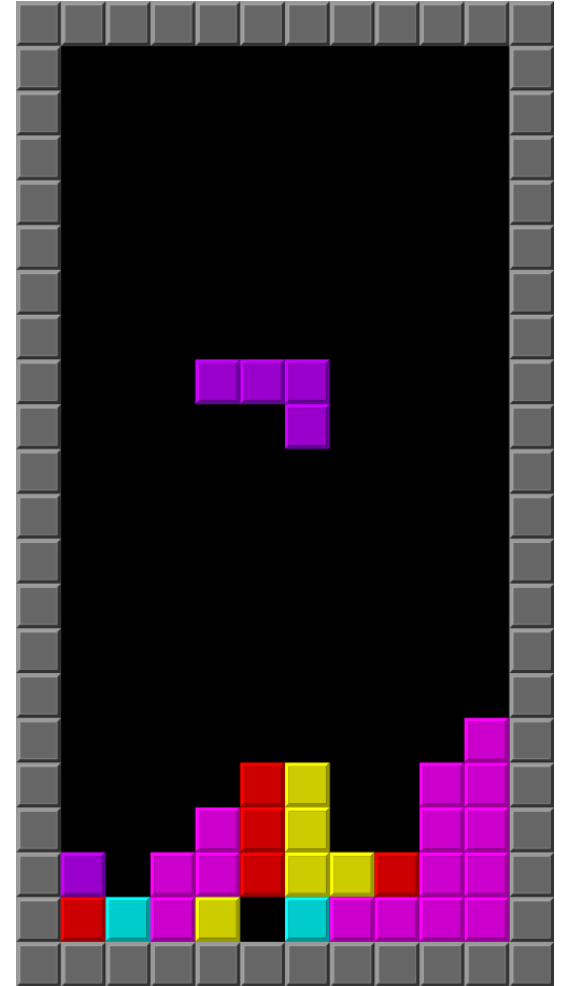
- ❖ Data structures in algorithmic design
- ❖ Stacks
- ❖ Queues

But first...



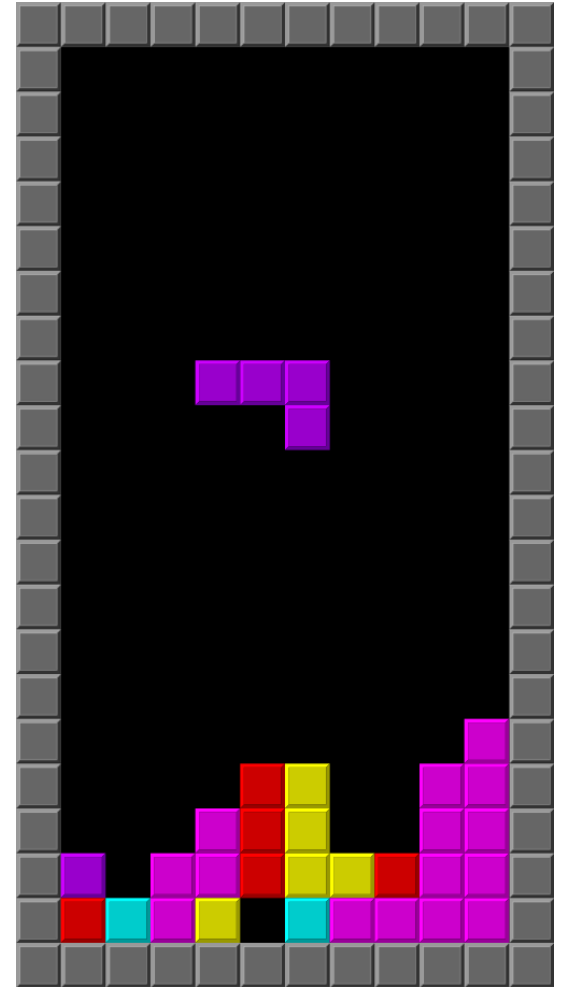
But first...

- ❖ Tetris: a tile-packing puzzle game



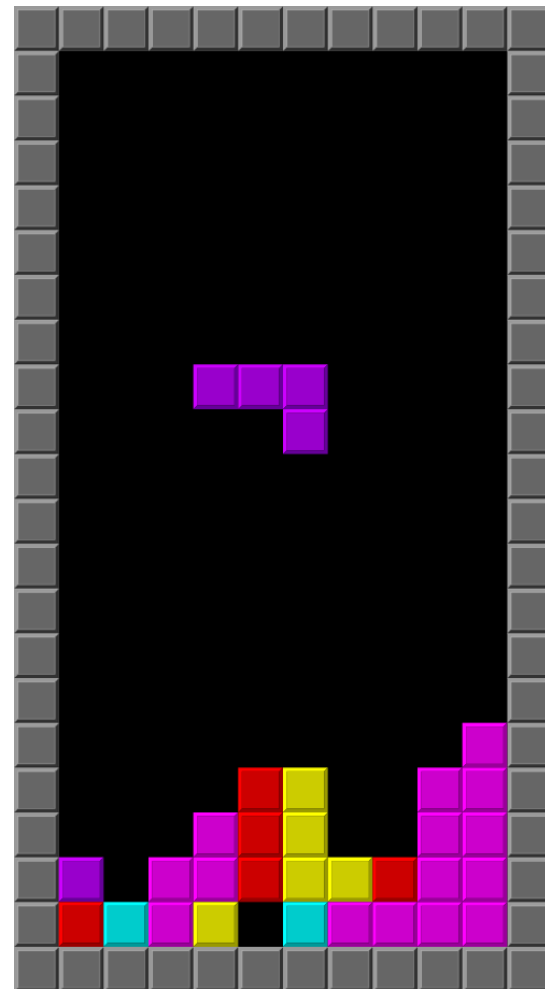
But first...

- ❖ Tetris: a tile-packing puzzle game
- ❖ Goal: densely pack the tiles



But first...

- ❖ Tetris: a tile-packing puzzle game
- ❖ Goal: densely pack the tiles
- ❖ Advanced strategies have the player organizing game pieces in particular ways and thinking many tiles ahead



Data Structures

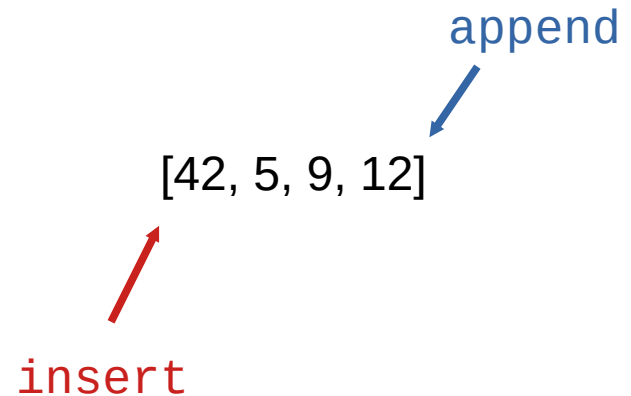
- ❖ How should we organize information to efficiently accomplish given tasks?

Data Structures

- ❖ How should we organize information to efficiently accomplish given tasks?
- ❖ Python built-in data structures:
 - lists, sets, tuples, dictionaries, ...
 - Good at some tasks and bad at others

Data Structures

- ❖ How should we organize information to efficiently accomplish given tasks?
- ❖ Python built-in data structures:
 - lists, sets, tuples, dictionaries, ...
 - Good at some tasks and bad at others
- ❖ Example: Lists in Python
 - append to back of list is $O(1)$
 - insert t in front of list is $O(n)$



Data Structures

- ❖ How should we organize information to efficiently accomplish given tasks?
- ❖ Use data structures that are tailored to our task
 - stacks, queues, heaps, graphs, maps, etc.
 - different implementations of familiar data structures

Abstract Data Types

- ❖ Data type defined solely by its behavior from the user's point of view
- ❖ List (abstract data type)
 - Stores items in order
 - Get/insert/remove item at any index

Abstract Data Types

- ❖ Data type defined solely by its behavior from the user's point of view
- ❖ List (abstract data type)
 - Stores items in order
 - Get/insert/remove item at any index
- ❖ Python's implementation of a list (sometimes called an array)
 - Maintains an index where each index value points to an object
 - $O(1)$ operations: getting value at index; appending to the end
 - $O(n)$ operations: inserting at index; removing at index

Task 1

We have some objects stored in order and want to efficiently do the following tasks (ideally in $O(1)$ time):

- ❖ **Push:** add an object to the front of the order
- ❖ **Pop:** remove an object from the front of the order
- ❖ **Peek:** look at the front object

Task 1

We have some objects stored in order and want to efficiently do the following tasks (ideally in $O(1)$ time):

- ❖ **Push**: add an object to the front of the order
- ❖ **Pop**: remove an object from the front of the order
- ❖ **Peek**: look at the front object

Does an array satisfy these requirements?

Task 1

We have some objects stored in order and want to efficiently do the following tasks (ideally in $O(1)$ time):

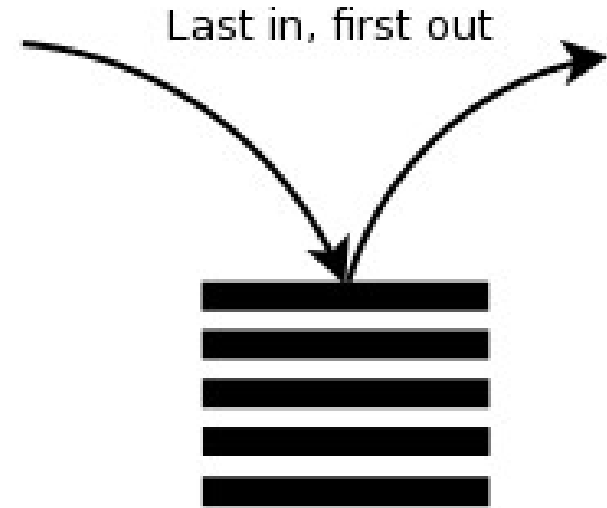
- ❖ **Push**: add an object to the front of the order
- ❖ **Pop**: remove an object from the front of the order
- ❖ **Peek**: look at the front object

Does an array satisfy these requirements?

- ❖ No, pushing and popping are both $O(n)$

Stack

- ❖ An abstract data type that holds objects in order and can
 - Push
 - Pop
 - Peek



Implementing a Stack

Strategy: build a stack out of data structures we know

- ❖ Implement a stack using an array (Python's list)

Implementing a Stack

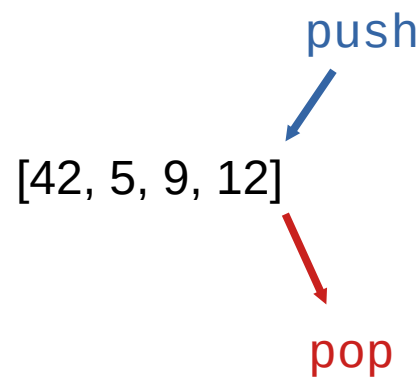
Strategy: build a stack out of data structures we know

- ❖ Implement a stack using an array (Python's list)
 - Holds objects in order and can push, pop, and peek
 - However: pushing and popping are linear time operations

Implementing a Stack

Strategy: build a stack out of data structures we know

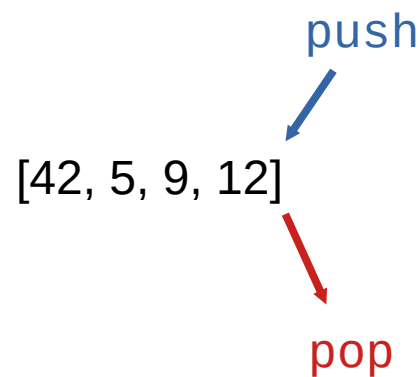
- ❖ Implement a stack using an array (Python's list)
 - Holds objects in order and can push, pop, and peek
 - However: pushing and popping are linear time operations
- ❖ Implement a stack using end of array instead



Implementing a Stack

Strategy: build a stack out of data structures we know

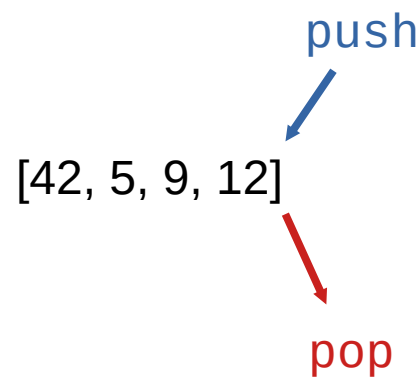
- ❖ Implement a stack using an array (Python's list)
 - Holds objects in order and can push, pop, and peek
 - However: pushing and popping are linear time operations
- ❖ Implement a stack using end of array instead
 - push is appending to the end: $O(1)$
 - pop is removing from end: $O(1)$
 - peek is getting at index: $O(1)$



Implementing a Stack

Strategy: build a stack out of data structures we know

- ❖ Implement a stack using an array (Python's list)
 - Holds objects in order and can push, pop, and peek
 - However: pushing and popping are linear time operations
- ❖ Implement a stack using end of array instead
 - push is appending to the end: $O(1)$
 - pop is removing from end: $O(1)$
 - peek is getting at index: $O(1)$
- ❖ ds2 library has efficient implementation: `Stack()`



Task 2

We have some objects stored in order and want to efficiently do the following tasks (ideally in $O(1)$ time):

- ❖ **Enqueue**: add an object to the back of the order
- ❖ **Dequeue**: remove an object from the front of the order
- ❖ **Peek**: look at the front object

Task 2

We have some objects stored in order and want to efficiently do the following tasks (ideally in $O(1)$ time):

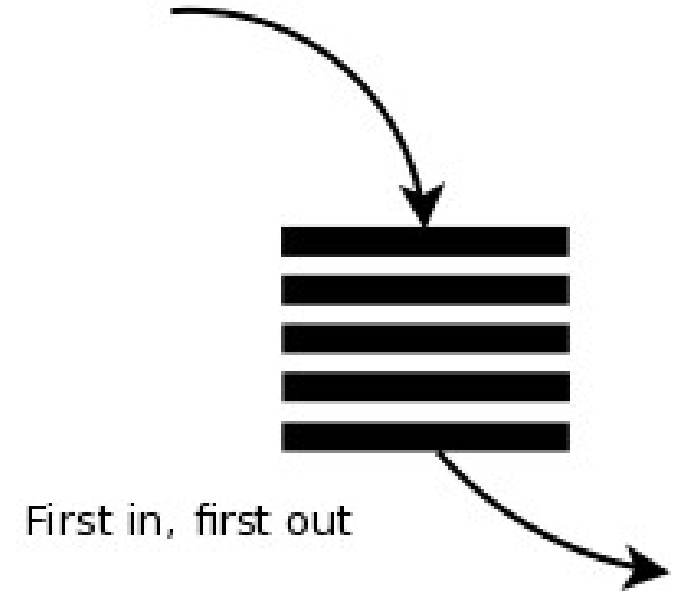
- ❖ **Enqueue**: add an object to the back of the order
- ❖ **Dequeue**: remove an object from the front of the order
- ❖ **Peek**: look at the front object

Does an array satisfy these requirements?

- ❖ No, dequeue is $O(n)$

Queue

- ❖ An abstract data type that holds objects in order and can
 - Enqueue
 - Dequeue
 - Peek



Implementing a Queue

Strategy: build a queue out of data structures we know

- ❖ Implement a queue using an array (Python's list)
 - Holds objects in order and can enqueue, dequeue, and peek
 - However: dequeue are linear time operations

Implementing a Queue

Strategy: build a queue out of data structures we know

- ❖ Implement a queue using an array (Python's list)
 - Holds objects in order and can enqueue, dequeue, and peek
 - However: dequeue are linear time operations
- ❖ Can efficiently implement a queue using a new type of list: linked list
 - We'll come back to this in a few weeks!

Implementing a Queue

Strategy: build a queue out of data structures we know

- ❖ Implement a queue using an array (Python's list)
 - Holds objects in order and can enqueue, dequeue, and peek
 - However: dequeue are linear time operations
- ❖ Can efficiently implement a queue using a new type of list: linked list
 - We'll come back to this in a few weeks!
- ❖ ds2 library has efficient implementation: `Queue()`

Next Time

- ❖ Graphs
- ❖ DSP Ch. 20